Project Report : FoSyMa

Member : Thomas LU

Git Repository Link :
https://github.com/CHOMUSKE/multiagent-graph-exploration-coordination

15 octobre 2026

Table of Contents

1 Introduction **2**

2 General Code Architecture **2**
2.1 Agent Abstraction and Specialization 2
2.2 Class Diagram 2
2.3 Hunt Orchestration : The Package `coalition_golem` 2
2.4 Behaviour Dynamics (*Behaviours*) 3
2.5 Environment Modeling and Utilities 3

3 Design Choices **3**
3.1 Hierarchical Exploration Strategy 3
3.1.1 Algorithmic Analysis 4
3.2 Communication Protocols and Knowledge Sharing 4
3.2.1 Sequential Exchange Protocol (Ping-Pong-ACK) 4
3.2.2 Knowledge Merging and Consistency 5
3.3 Coordination and Hunting Strategy 5
3.3.1 Localization by Scent Trails (Protocol `smell`) 5
3.3.2 Coalition Infrastructure : `flood_optimisé` 5
3.3.3 Encirclement Mechanics and Blocking Condition 6
3.3.4 State Machine Dynamics (FSM) 6

4 Algorithm Analysis **6**
4.1 Metric Details 7
4.2 System Strengths and Limitations 7
4.2.1 Strengths 7
4.2.2 Limitations and Future Directions 8

5 Conclusion **8**
5.1 Summary 8
5.2 Critical Review and Possible Improvements 8

1 Introduction

This report presents the development of a multi-agent system capable of exploring an unknown environment and coordinating a hunt for the Golem...

2 General Code Architecture

The system architecture was designed to combine operational robustness with software extensibility. It is organized around a strict separation between knowledge management, decision-making logic, and communication protocols.

2.1 Agent Abstraction and Specialization

At the top of the hierarchy is the `AbstractCommunicationAgent` class. This abstract class centralizes all attributes required for exploration and tracking missions.

This abstraction choice is intended to ensure optimal extensibility : implementing future agent types is greatly simplified. As an example, I developed the `ControlledAgent2` class. Acting as a deliberately static sentinel, it served as a deterministic experimental subject for accurately validating map-sharing protocols (*share-map*) and the correct receipt of operational instructions, without interference from random movements.

2.2 Class Diagram

The following diagram (Figure 1) details the structure of the agents and their respective behaviours.

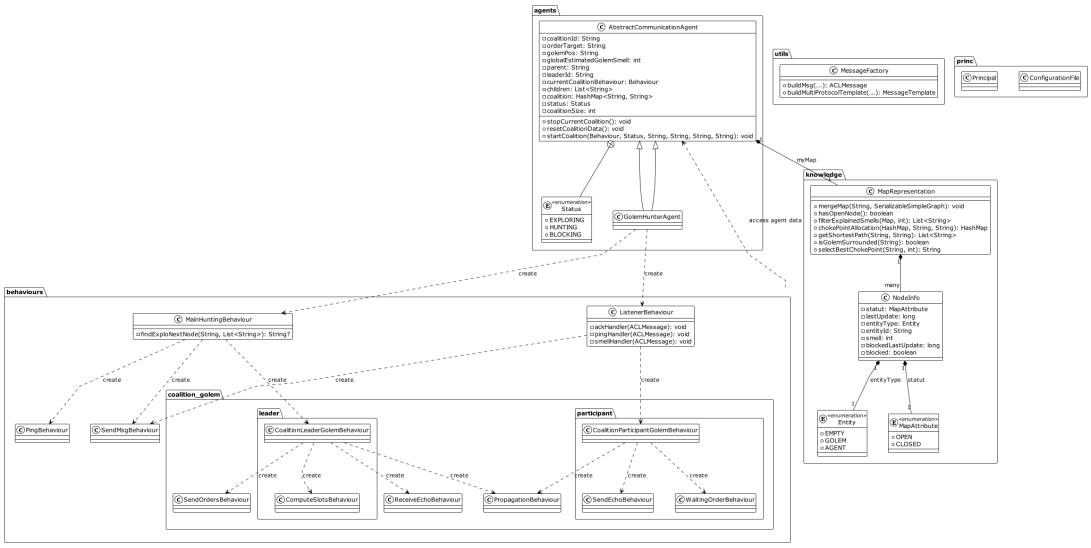


FIGURE 1 – Modular Agent-Behaviour Architecture

2.3 Hunt Orchestration : The Package `coalition_golem`

The coordination logic, the core of the hunt, is contained within the `coalition_golem` package. The group's collective intelligence is modeled there by a **Finite State Machine**

(FSM), ensuring a smooth and rigorous transition between the different mission phases. This package is structured as follows :

- **Leader Components** : Two classes dedicated to strategic decision-making and role allocation.
- **Participant Components** : Three classes governing order execution and close-range tracking.
- **Shared Foundation** : Three cross-cutting classes ensuring functional consistency between the two roles.

2.4 Behaviour Dynamics (*Behaviours*)

The agents' life cycle is based on a duality between passive monitoring and proactive action :

- `ListenerBehaviour` : This behaviour provides constant information monitoring. Running asynchronously in the background (without a specific trigger), it intercepts unexpected messages. It acts as a catalyst capable of activating critical triggers, such as initializing the participant state machine when an external request is received.
- `MainHuntingBehaviour` : It forms the agent's decision-making core. This class orchestrates the main movement routines and empirical analysis of the immediate environment through direct observation.
- **Exchange Abstraction** : To keep the code clean, the `PingBehaviour` (of type *OneShot*) and `SendMsgBehaviour` classes were implemented. They encapsulate the complexity of messaging protocols, avoiding redundant, cumbersome code blocks and ensuring standardized sending parameters.

2.5 Environment Modeling and Utilities

Topological management is handled by the `MapRepresentation` class. Acting as the spatial knowledge manager, it updates the graph (`addNodes`), strategically allocates hunting positions, and calculates routes using **weighted Dijkstra** algorithms.

To enrich this representation, the `NodeInfo` class was introduced to aggregate multiple semantic descriptors for each node. Finally, `MessageFactory`, an implementation of the design pattern of the same name, centralizes message creation and reception filters (*templates*), thereby improving development speed and maintenance of the communication system.

3 Design Choices

3.1 Hierarchical Exploration Strategy

The agents' exploration process relies on decision-making structured into three priority levels, ensuring optimal graph coverage and continuous movement.

1. **Topological Exploration (High Priority)** : The agent prioritizes resolving "open nodes." The destination is selected using a **weighted Dijkstra** algorithm.
   - **Dynamic weighting** : The cost of an edge is adjusted according to the entities present on the nodes.

— **Obstruction handling** : If a calculated path crosses a cell marked as `blocked`, the algorithm invalidates the route (returns `null`), forcing the agent to recalculate a viable alternative.

2. **Temporal Monitoring (Medium Priority)** : When there are no new nodes to discover, the agent switches to a refresh strategy. It targets the nearest node whose last update (`lastUpdate`) was more than 20 seconds ago. This approach is crucial for locating the Golem in areas that have already been explored but may have changed due to its movements.
3. **Stochastic Behaviour (Low Priority)** : If none of the previous criteria is met, the agent makes a random move. This safeguard prevents the agent from becoming stationary and unintentionally blocking graph bottlenecks for other coalition members.

3.1.1 Algorithmic Analysis

The efficiency of the movement strategy relies on the `getShortestPath` method, which goes beyond simple topological calculation by integrating semantic constraints in real time.

— **Time complexity** : Path calculation is performed in $O(V \log V + E)$, where V represents the number of nodes in the known graph and E the number of edges. This complexity includes :
  — A preprocessing phase in $O(E)$ to dynamically update edge weights according to node occupancy by other entities.
  — Execution of Dijkstra's algorithm using these weights to identify the lowest-cost route.
  — A path extraction and filtering phase in $O(V)$.
— **Optimality and Robustness** : The algorithm is optimal with respect to the cost function defined by the presence of agents (a cost of 10.0 for fresh information) or obstacles. Unlike a conventional breadth-first search, the implementation provides strategic safety : if the total path weight exceeds a critical threshold (1000.0), the path is considered obstructed and the method returns `null`. This makes it possible to intelligently avoid areas with a high density of agents or tactical dead ends before movement even begins.

3.2 Communication Protocols and Knowledge Sharing

Consistency in the global view of the environment relies on an optimized map-data exchange protocol designed to reduce network load while maximizing information freshness.

3.2.1 Sequential Exchange Protocol (Ping-Pong-ACK)

Knowledge sharing follows a four-step life cycle, triggered periodically (every three execution cycles) or by direct detection of neighbours :

1. **Discovery Phase (Ping)** : The agent sends a *Ping* message by *broadcast* or directly to neighbours identified through direct observation. This message contains a global estimate of the Golem's scent (`smellGlobalEstimation`). Although this data is related to hunting, including it in the *Ping* is a "piggybacking" optimization that propagates strategic information without generating additional messages.

2. **Synchronization (Pong)** : Upon receipt, agents have a 5-second time window to signal their presence with a *Pong* message.
3. **Differential Transfer (Share-Map)** : To avoid redundantly sending the entire map, the sender uses a `HashMap` structure to store the date of the last exchange with each collaborator. The `createSubGraph` method is then called to generate a subgraph containing only nodes whose update date (`lastUpdate`) is later than the last contact.
4. **Validation and Update (ACK)** : Once the subgraph has been received and integrated using the `mergeMap` method, the recipient sends an *ACK*. This final signal allows the original sender to update its temporal database for future exchanges.

3.2.2 Knowledge Merging and Consistency

Integrating remote data through the `mergeMap` method is not simple concatenation, but a conflict-resolution process based on two criteria :

— **Temporal Criterion** : Information is updated only if the incoming timestamp (`lastUpdate` or `blockedLastUpdate`) is strictly later than the one stored locally.
— **Hierarchical Criterion** : Node status follows a priority rule that prevents, for example, a node marked as closed (`closed`) from being reopened by obsolete or lower-priority information.

3.3 Coordination and Hunting Strategy

The tracking phase is the culmination of the mission and relies on a carefully managed transition between detecting environmental clues and engaging a coordinated coalition.

3.3.1 Localization by Scent Trails (Protocol `smell`)

Identifying the Golem's position initially relies on detecting scent trails. The scent perceived by the sensors does not provide an exact coordinate, but rather a range that makes it possible to estimate the Golem's area of presence.

— **Transition to the *Smelling* State** : As soon as a scent is detected, the agent switches from the *exploring* state to the *smelling* state. A `smell` protocol message is then broadcast, containing the list of nodes where scents were identified.
— **Scent Analysis and Filtering** : To distinguish a free Golem from one that has already been neutralized, the system uses the `filterExplainedSmells` method. This compares observed emissions with the positions of Golems already surrounded within a given radius (`radius`). If the number of detected scents exceeds the number of Golems identified as blocked, the information is considered suspicious (`suspiciousSmells`), triggering a priority investigation.

3.3.2 Coalition Infrastructure : `flood_optimisé`

Direct detection of the Golem triggers initialization of `CoalitionLeaderGolemBehaviour`, with the initiating agent becoming the hunt *Leader*. Coalition recruitment relies on an **optimized flood** mechanism :

— **Filtering and Anti-Spam** : The `ListenerBehaviour` intercepts `flood_golem` messages. By using a `conversation_id`, any message already processed is ignored, immediately stopping redundant propagation and preserving bandwidth.

— **Leadership Conflict Management** : When several potential leaders are present, delegation is controlled based on agent identifiers. An agent agrees to join a new coalition only if the sending leader has a higher priority, thereby ensuring order stability.

3.3.3 Encirclement Mechanics and Blocking Condition

The capture strategy aims to immobilize the target entity completely by occupying its immediate neighbourhood.

— **Target Allocation (*Slots*)** : The leader defines the Golem's position as the central blocking point (*chokepoint*). It then allocates adjacent nodes to team members according to their respective proximity.

— **Blocking Prediction Using Weighted Dijkstra** : The use of **weighted Dijkstra** plays a crucial role in encirclement. By dynamically increasing the cost of nodes occupied by allies, the algorithm encourages participants to converge on the Golem via distinct paths. This behaviour creates a natural pincer movement, closing off the Golem's escape routes simultaneously.

— **Blocking Validation** : Capture is confirmed as successful when every node neighbouring the Golem's position is either occupied by an entity or marked as `blocked` by the environment's topology.

3.3.4 State Machine Dynamics (FSM)

The maneuver is synchronized by two finite state machines :

— **The Leader (`CoalitionLeaderGolemBehaviour`)** : It orchestrates signal dissemination, collection of responses (*echoes*), calculation of node assignments, and transmission of final positioning commands.

— **The Participants (`CoalitionParticipantGolemBehaviour`)** : They relay the flood signal, report the availability of their branch of the propagation tree, and execute the movement command as soon as they receive their assigned target.

4 Algorithm Analysis

This section details the theoretical performance of the implemented routines. The metrics used are **T** complexity (Time/local computation) and **C** complexity (Communication/network exchanges).

Algorithm	Complexity (T/C)	Optimality	Stopping criterion
Exploration	T : $O(V \log V + E)$ C : $O(N)$ par cycle	Yes (complete)	No `open_nodes` and refresh completed
Hunting	T : $O(N_{agents} \times$ Dijkstra) C : $O(N)$ (flood optimisé)	Heuristic	Golem encerclé (voisins `blocked` ou agents)
Communication	T : $O(E)$ (prétraitement) C : $O(V_{diff} + E_{diff})$	Optimal (differential)	Réception du message de validation `ACK`

TABLE 1 – Summary analysis of the implemented algorithms

4.1 Metric Details

— **Exploration** : Time complexity is determined by the weighted Dijkstra algorithm ($O(V \log V + E)$) used to target open nodes. Termination is confirmed when no accessible or unvisited nodes remain.
— **Hunting** : The leader assigns targets among the nodes adjacent to the Golem. Success is defined as completely immobilizing the entity through a combination of agents and natural obstacles.
— **Communication** : The protocol minimizes network load by transmitting only graph changes (*subgraph*) since the last recorded exchange.

4.2 System Strengths and Limitations

The system evaluation highlights a balance between communication protocol efficiency and certain topological constraints related to space occupancy.

4.2.1 Strengths

The system stands out for its optimized resource management and structural robustness :

— **Communication Efficiency** : Using the `createSubGraph` method and differential transfer drastically reduces bandwidth consumption. The network is never saturated by redundant transmissions.
— **Scalability** : The `flood_optimisé` mechanism, including filtering by `conversation_id`, makes it possible to coordinate a large number of agents without an explosion in network traffic.
— **Pathfinding Intelligence** : Integrating **weighted Dijkstra** elegantly addresses collision avoidance and encirclement. The pincer behaviour emerges without complex geometric calculations.
— **Resilience and Decentralized Autonomy** : The system is highly robust in the face of unexpected unit loss. The individual autonomy of agents ensures that no failure causes a critical blockage for the rest of the fleet. Whether an agent is lost during a map exchange, coalition formation, or blocking, the remaining agents

continue their objectives without interruption. This resilience also applies to the leader : its disappearance is not fatal to the mission.

4.2.2 Limitations and Future Directions

Despite its robustness, the current approach has potential bottlenecks :

— **The "Gatekeeper" Dilemma and Partitioning** : A major limitation is that agents remain stationary after a successful capture. If this encirclement occurs at a bottleneck, it can partition the graph into two disconnected areas.

— **Single-Golem Interpretation** : Agents currently see only one Golem on their map, which can sometimes cause them to abandon an almost completed hunt to pursue another Golem while passing near another coalition.

5 Conclusion

5.1 Summary

This project made it possible to design and implement a decentralized multi-agent system capable of operating efficiently in a dynamic environment that is initially unknown. The success of the hunt relies on the synergy of three technological pillars :

— **Intelligent Map Management** : Using the **weighted Dijkstra** algorithm made it possible to combine navigation, collision avoidance, and encirclement strategy into a single mechanism. By dynamically adjusting edge costs, the agents produce an emergent pincer behaviour.

— **Lean and Robust Communication** : The differential exchange protocol (Ping-Pong-ACK), using the `createSubGraph` method, ensures consistency of shared knowledge while minimizing network load by sending only temporal updates.

— **Structured Coordination** : Implementing finite state machines (FSMs) within the `coalition_golem` package ensured a rigorous transition between individual exploration and collective tracking, while managing leadership conflicts through delegation.

5.2 Critical Review and Possible Improvements

Post-development analysis reveals areas for optimization to improve the fleet's overall performance :

— **Resolving Bottlenecks** : The graph-partitioning problem caused by permanent blocking is the main limitation of the current system. A major improvement would be to implement a **right-of-way handover** protocol or circular swapping of agent positions, maintaining map connectivity without releasing the target.

— **Multi-Target Arbitration** : The current tendency to focus on a single Golem could be corrected through an inter-leader communication system. This would prevent a coalition from being abandoned in favour of a less advanced hunt, thereby optimizing overall capture time.

In conclusion, although challenges remain in the fine-grained management of spatial resources, the modular architecture and implemented navigation algorithms provide a solid foundation for deploying effective collective intelligence.